d) Authentication tokens are tied to an individual user identity, while capability tokens can be shared between users

e) Authentication tokens are short-lived, while capabilities often have a longer lifetime

9.2.2 Using capability URIs in the Natter API

To add capability URIs to Natter, you first need to implement the code to create a capability URI. To do this you can reuse an existing `TokenStore` implementation to create the token component, encoding the resource path and permissions into the token attributes as shown in listing 9.1. Because capabilities are not tied to an individual user account, you should leave the username field of the token blank. The token can then be encoded into the URI as a query parameter, using the standard `access_token` name from RFC 6750. You can use `request.url()` in Spark to get the original URL the client used to connect to the API to use as the base URI, and then use the `resolve()` method on that to construct a new URI with the correct hostname and port. Open the Natter API project[37] and navigate to src/main/java/com/manning/apisecurityinaction/controller and create a new file named CapabilityController.java with the content of listing 9.1 and save the file.

> **TIP** Because you'll be switching the Natter API to use capability URLs as the primary approach to access control, you should make sure the tokens never expire